



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Lab Report No:-8

Title:- Disk scheduling Algorithms

Submitted by:-

Name:- Surajan Shrestha

Roll No:- 38

Submitted To:- Drish mali

Department Of CSIT

Checked by:-

Date of submission:- 2081/06/08

TITLE : Disk scheduling Algorithms

OBJECTIVE :

→ The objective of this lab is to implement and understand different disk scheduling algorithms like FCFS, SSTF and SCAN. These algorithms help in determining the movement of the disk's read/write head and minimize the total head movement to optimize performance.

THEORY :

→ Disk scheduling algorithms manage the order in which read/write requests are processed by the disk's head to minimize seek time, enhance throughput and optimize response time.

1, FCFS (first come first serve) :

→ FCFS serves disk I/O requests in the order they arrive. It is the simplest disk scheduling algorithm, but it is not always the most efficient in terms of minimizing total head movement.

2, SSTF (Shortest seek Time first) :

→ SSTF selects the disk request closest to the current head position, regardless of the order in which requests arrive. While SSTF improves over FCFS by reducing head movement, it can lead to starvation for requests far from the current head position.

3, SCAN (Elevator Algorithm) :

→ In SCAN, the disk's head moves in one direction, servicing requests until it reaches the end of

isk, then reverses direction and services requests on the return journey. This algorithm ensures a more uniform wait time and minimizes the total head movement by sweeping across the disk in both directions.

OBSERVATION :

// FCFS Disk scheduling

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int RQ[100], i, n, TotalHeadMoment = 0, Initial;
```

```
    printf("Enter the number of Requests \n");
```

```
    scanf("%d", &n);
```

```
    printf("Enter the Requests sequence \n");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf("%d", &RQ[i]);
```

```
    printf("Enter initial head position \n");
```

```
    scanf("%d", &Initial);
```

```
    for (i = 0; i < n; i++) {
```

```
        TotalHeadMoment = TotalHeadMoment + abs(RQ[i] - Initial);
```

```
        Initial = RQ[i];
```

```
    }
```

```
    printf("Total head moment is %d", TotalHeadMoment);
```

```
    return 0;
```

```
}
```

sample output:

Enter the number of Requests : 4
Enter the Requests sequence : 95 180 34 119
Enter initial head position : 50
Total head movement is 642

SSTF Disk scheduling

```
#include <stdio.h>
#include <limits.h>
#include <stdlib.h>
using namespace std;

int main()
{
    int RQ[100], i, n, TotalHeadmovement = 0, initial, count = 0;

    printf("Enter the number of Requests \n");
    scanf("%d", &n);

    printf("Enter the requests sequence \n");
    for (i = 0; i < n; i++)
        scanf("%d", &RQ[i]);

    printf("Enter initial head position \n");
    scanf("%d", &initial);

    while (count < n) {
        int min = INT_MAX, d, index;
        for (i = 0; i < n; i++) {
            d = abs(RQ[i] - initial);
            if (min > d) {
                min = d;
                index = i;
            }
        }
        initial = RQ[index];
        TotalHeadmovement += d;
        count++;
    }

    printf("Total head movement is %d", TotalHeadmovement);
}
```

```

TotalHeadMoment += min;
initial = RQ[Index];
RQ[Index] = INT_MAX;
count++;
}

```

```

printf("Total head movement is %d \n", TotalHeadMoment);
return 0;
}

```

INPUT:

```

Enter the number of Requests : 5
Enter the Requests sequence :
55 58 59 18 90
Enter initial head position :
50
Total head movement is 158

```

SCAN DISK scheduling

```

#include <stdio.h>
#include <stdlib.h>

```

```

int main() {
    int RQ[100], i, j, n, TotalHeadMoment = 0, initial,
    size, move;
    printf("Enter the number of Requests \n");
    scanf("%d", &n);
    printf("Enter the Requests sequence \n");
    for (i = 0; i < n; i++)
        scanf("%d", &RQ[i]);
    printf("Enter initial head position \n");
}

```



```

scanf("%d", &initial);
if ("Enter total disk size \n");
scanf("%d", &size);
if ("Enter the head movement direction for
high(1) and for low(0) \n");
scanf("%d", &move);
(i=0; i<n; i++) {
(j=0; j<n-i-1; j++) {
if (RQ[j] > RQ[j+1]) {
int temp = RQ[j];
RQ[j] = RQ[j+1];
RQ[j+1] = temp;
}
}
}

```

```

= index;
for (i=0; i<n; i++) {
if (initial < RQ[i]) {
index = i;
break; }
}

```

```

(move == 1) {
or (i = index; i < n; i++) {
Head moment = Total Head moment + abs(RQ[i] -
initial);
initial = RQ[i]; }
}

```

TotalHeadMoment = TotalHeadMoment + abs(size -
[i-1]-1);

initial = size - 1;

{ (i = index - 1; i >= 0; i--) {

TotalHeadMoment = TotalHeadMoment + abs(RQ[i] -
initial);

initial = RQ[i];

else {

for (i = index - 1; i >= 0; i--) {

TotalHeadMoment = TotalHeadMoment + abs(RQ[i] -
initial);

initial = RQ[i]; }

TotalHeadMoment = TotalHeadMoment + abs(RQ[i+1] - 0);

initial = 0;

for (i = index; i < n; i++) {

TotalHeadMoment = TotalHeadMoment + abs(RQ[i] -
initial);

initial = RQ[i];

}

printf("Total head movement is %d", TotalHeadMoment);
return 0;

}

OUTPUT:

Enter the number of requests : 4

Enter Requests sequence : 95 180 34 119

Enter initial head position : 50

Enter total disk size : 200

Enter the head movement direction for high(1) and low(0) : 1

Total head movement is 391.

DISCUSSION:

→ In this lab, three different disk scheduling algorithms - FCFS, SSTF and SCAN - were implemented and compared in terms of total head movement. The FCFS algorithm handles requests in order they arrive, which can lead to high total head movement, especially when requests are scattered. The SSTF algorithm reduces head movement by serving the request closest to current head position, but it can cause starvation for distant requests. The SCAN handles requests in a sweeping manner, moving head in one direction & then reversing, which balances efficiency with fairness.

CONCLUSION:

→ The lab demonstrates that FCFS is simple but can result in higher head movement, SSTF reduces head movement but may cause the starvation, while SCAN provides a balanced approach, optimizing both efficiency and fairness in disk scheduling.